

이진트리(Binary Tree) 정리

1. 트리(Tree) 개요

- **정의:** 계층적(계통적) 구조를 가지는 비선형 자료구조
- **구성 요소:** 노드(Node)와 간선(Edge)
- **특징:**
 - 하나의 루트 노드(root node) 존재
 - 순환(cycle)이 없음
 - 노드 간 관계는 부모-자식 구조

2. 이진트리(Binary Tree)

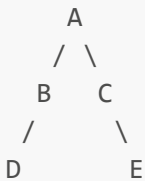
정의

- 모든 노드가 **최대 2개의 자식 노드**를 가지는 트리
- 자식 노드는 **왼쪽(left child)**, **오른쪽(right child)**으로 구분
- 각 노드는 **0개, 1개, 2개의 자식 노드**를 가질 수 있음

재귀적 정의

- **빈 트리(NULL)** 또는
- **루트 노드 + 왼쪽 이진트리 + 오른쪽 이진트리**

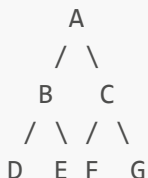
예시



3. 이진트리의 종류

1) 포화 이진트리 (Full Binary Tree)

- 모든 내부 노드가 **정확히 2개의 자식**을 가짐
- 모든 리프 노드가 **같은 레벨**에 존재



- 노드 수: $(2^h - 1)$, h 는 트리의 높이

2) 완전 이진트리 (Complete Binary Tree)

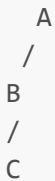
- 마지막 레벨을 제외한 모든 레벨이 **노드로 꽉 차 있음**
- 마지막 레벨은 **왼쪽부터 채워짐**



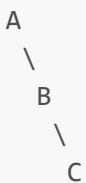
3) 편향 이진트리 (Skewed Binary Tree)

- 모든 노드가 한쪽 자식만 가짐
- **왼쪽 편향 or 오른쪽 편향**

왼쪽 편향 예시:



오른쪽 편향 예시:



4. 구현 방식

1) 배열(Array) 기반 구현

- **포화 이진트리**로 가정하고 인덱스로 노드를 배치
- 인덱스 관계:
 - 부모: i
 - 왼쪽 자식: $2*i$
 - 오른쪽 자식: $2*i + 1$

예:

배열: [_, A, B, C, D, E, F] // 인덱스 1부터 시작
트리:

```

      A(1)
     /  \
    B(2)  C(3)
   / \   /
  D(4) E(5) F(6)

```

2) 연결리스트(Linked List) 기반 구현

```

struct Node {
    int data;
    Node* left;
    Node* right;
};

```

- 동적 메모리 할당과 포인터를 통해 노드 연결
- 각 노드는 자신이 가리키는 왼쪽/오른쪽 자식 노드를 포인터로 연결

5. 이진트리 순회(Tree Traversal)

- 트리의 모든 노드를 체계적으로 방문하는 방법
- 세 가지 기본 방식 존재

1) 전위순회 (Pre-order)

- 순서: 루트 → 왼쪽 → 오른쪽
- 응용: 트리 구조 저장, 수식 변환

```

void preorder(Node* node) {
    if (!node) return;
    cout << node->data << ' ';
    preorder(node->left);
    preorder(node->right);
}

```

2) 중위순회 (In-order)

- 순서: 왼쪽 → 루트 → 오른쪽
- 응용: 중위표기식(infix) 출력

```
void inorder(Node* node) {
    if (!node) return;
    inorder(node->left);
    cout << node->data << ' ';
    inorder(node->right);
}
```

3) 후위순회 (Post-order)

- 순서: 왼쪽 → 오른쪽 → 루트
- 응용: 트리 소거, 수식 계산

```
void postorder(Node* node) {
    if (!node) return;
    postorder(node->left);
    postorder(node->right);
    cout << node->data << ' ';
}
```

예시 트리 순회 결과

A

/ \

B C

/ \

D E

전위: A B D E C
중위: D B E A C
후위: D E B C A

6. 요약

구분	정의	최대 노드 수 (높이 h)	특징
포화 이진트리	모든 노드가 자식 2개	$(2^h - 1)$	모든 노드가 가득참
완전 이진트리	마지막 레벨만 왼쪽부터 채움	$(2^h - 1)$ 이하	배열 표현에 유리함
편향 이진트리	한쪽 자식만 존재	h개	메모리 낭비 많음

7. 활용 예시

- 컴파일러 구성: 문법 트리(AST)
- 수식 평가기: 후위 표기법 평가

- **메모리 관리:** 힙 트리(Heap Tree)
 - **탐색 최적화:** 이진 탐색 트리(BST), AVL 트리 등
-

이진트리는 컴퓨터 과학의 기본 중 하나로, 알고리즘 설계, 시스템 구현, 데이터 처리에서 광범위하게 활용됩니다. 다양한 유형과 순회 방식을 이해하고 구현해보는 것이 중요합니다.